

HEALTH CHAIN: INTEGRATING MYSQL WITH BLOCKCHAIN FOR RESILIENT HEALTHCARE SYSTEMS

A PROJECT REPORT

Submitted by

GAUTHAM R (720922104024)

HARISH B (720922104029)

KARTHICK R (720922104303)

KISHORE R V (720922104305)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

JCT COLLEGE OF ENGINEERING AND TECHNOLOGY

Pichanur, Coimbatore

ANNA UNIVERSITY : CHENNAI 600 025

MARCH 2026

**JCT COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)
COIMBATORE - 641105**

BONA FIDE CERTIFICATE

Certified that this project report “**HEALTH CHAIN: INTEGRATING MYSQL WITH BLOCKCHAIN FOR RESILIENT HEALTHCARE SYSTEMS**” is the bonafide work of “**GAUTHAM R, HARISH B, KARTHICK R, KISHORE R V**” who carried out the project work under my supervision.

Submitted for the Viva Voce examination held on _____

SIGNATURE

SIGNATURE

Dr. P. D. R. Vijayakumar

Ms. Annal Sheeba Rani E

HEAD OF THE DEPARTMENT

SUPERVISOR

**Department of Computer Science and
Engineering**

**Department of Computer Science and
Engineering**

**JCT College of Engineering and Tech-
nology, Pichanur, Coimbatore - 641105**

**JCT College of Engineering and Tech-
nology, Pichanur, Coimbatore - 641105**

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that this project report entitled “**HEALTH CHAIN: INTEGRATING MYSQL WITH BLOCKCHAIN FOR RESILIENT HEALTH-CARE SYSTEMS**” is a record of original work carried out by us under the guidance of **Ms. Annal Sheeba Rani E, Assistant Professor**, Department of Computer Science and Engineering, JCT College of Engineering and Technology, Coimbatore, and has not formed the basis for the award of any degree, diploma, associateship, fellowship, or any other similar title.

GAUTHAM R (720922104024)

HARISH B (720922104029)

KARTHICK R (720922104303)

KISHORE R V (720922104305)

ABSTRACT

Health Chain is a full-stack Electronic Health Record system that combines a relational database workflow with an in-application blockchain ledger to provide tamper-evident clinical data management. The platform is designed to retain the practical benefits of a MySQL-based healthcare system while introducing a deterministic integrity layer that detects unauthorized modification of patient records.

The implemented architecture consists of a React frontend, an Express-based backend, a normalized MySQL schema, role-based access control with JWT, and a blockchain model persisted to JSON. During record creation, structured SOAP data is inserted transactionally into visit, vitals, diagnoses, and prescriptions tables. A deterministic clinical text representation is then hashed using SHA-256, and the hash is appended as a block linked to the previous block hash. Verification endpoints recompute the current hash from database state and compare it against the recorded blockchain hash to identify secure or tampered records.

The project additionally integrates operational healthcare requirements including appointment scheduling, doctor administration, patient timeline retrieval, and HIPAA-oriented audit logging. Security controls include password hashing, role-based authorization, optional transport security, and field-level encryption for sensitive profile attributes. Experimental walkthroughs confirm that while normal workflows remain responsive, direct database tampering is reliably detected by the blockchain verification pipeline. The result is a practical prototype that demonstrates how blockchain-assisted integrity can be integrated into conventional healthcare software without replacing the underlying relational model.

ACKNOWLEDGEMENT

We express our sincere gratitude to **Dr. P. D. R. Vijayakumar**, Head of the Department, Department of Computer Science and Engineering, JCT College of Engineering and Technology, for his encouragement and institutional support during this project.

We are deeply thankful to our guide, **Ms. Annal Sheeba Rani E, Assistant Professor**, for her valuable guidance, constructive feedback, and continuous motivation throughout the development of this work.

We thank all faculty members, classmates, and well-wishers who contributed directly or indirectly to the successful completion of this project report.

CHAPTER NO.	TITLE	PAGE
	NO.	

TABLE OF CONTENTS

ABSTRACT	i
ACKNOWLEDGEMENT	ii
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF SYMBOLS AND ABBREVIATIONS	ix
1 INTRODUCTION	1
1.1 Background and Need	1
1.2 Problem Statement	2
1.3 Project Objectives	2
1.4 Scope and Boundaries	3
1.4.1 Included in Scope	3
1.4.2 Out of Scope	3
1.5 Contributions of the Work	3
1.6 Report Organization	4
2 LITERATURE SURVEY	5
2.1 Traditional EHR Systems and Integrity Challenges	5
2.2 Blockchain in Healthcare Data Management	5
2.3 Hash-Based Tamper Detection Approaches	6
2.4 Audit Logging and Compliance Perspective	6
2.5 Research Gap and Motivation	7

3	PROJECT REQUIREMENTS AND SPECIFICATION	8
3.1	Functional Requirements	8
3.2	Non-Functional Requirements	9
3.3	Input-Output Contracts	9
3.4	Data Model Overview	10
4	SYSTEM ARCHITECTURE AND DESIGN	11
4.1	High-Level Architecture	11
4.2	Backend Module Design	11
4.3	Record Creation and Verification Flow	12
4.4	Security and Compliance Design	12
5	DATABASE DESIGN AND DATA FLOW	14
5.1	Database Overview	14
5.2	Core Relational Schema Design	15
5.3	Transactional Database Reliability	15
5.4	Database Relationship and Access Path	16
5.5	Data Flow from Record Creation to Verification	17
5.6	Database and Data-Flow Summary	18
5.7	Implementation Flow Diagram	18
6	SOFTWARE ENGINEERING IMPLEMENTATION WORKFLOW	20
6.1	Engineering Environment and Stack Composition	20
6.2	Backend Engineering Workflow	21
6.2.1	Controller-Oriented Business Modules	21
6.2.2	Codebase Organization and Reuse Strategy	22
6.2.3	Reliability and Consistency Engineering	22
6.2.4	Failure Handling and Recovery Paths	22
6.3	Cross-Layer Integration Flow	22
6.4	Engineering Discussion	24
7	LOGIC DESIGN AND INTEGRITY ENGINE	25
7.1	Chapter Purpose and Logic Scope	25
7.2	Deterministic SOAP Canonicalization Logic	25

7.3	Hash Generation and Block Anchoring Logic	26
7.4	Verification Decision Engine	26
7.5	Role and Access Decision Logic	27
7.6	Scheduler Conflict Decision Logic	28
7.7	Error and Recovery Logic	28
7.8	Logic Validation Summary	28
8	SECURITY, PRIVACY, AND COMPLIANCE ANALYSIS	30
8.1	Security Objectives	30
8.2	Threat Model Overview	30
8.3	Authentication and Authorization Controls	31
8.4	Confidentiality and Data Protection Controls	31
8.5	Integrity and Non-Repudiation Perspective	32
8.6	Audit and Compliance Perspective	32
8.7	Limitations of Current Security Scope	33
8.8	Security Enhancement Roadmap	33
9	DEPLOYMENT, TESTING, EVALUATION, AND DISCUSSION	34
9.1	Deployment Overview	34
9.2	Configuration Checklist	34
9.3	Demo Data and Reset Workflow	35
9.4	Testing, Evaluation, and Discussion Overview	36
9.5	Testing Methodology	36
9.6	Representative Validation Matrix	36
9.7	Tamper-Evidence Evaluation Scenario	37
9.8	Discussion of Results	38
9.8.1	Positive Outcomes	38
9.8.2	Observed Engineering Constraints	38
9.8.3	Evaluation Summary	38
10	CONCLUSION AND FUTURE ENHANCEMENTS	40
10.1	Conclusion	40
10.2	Limitations	41
10.3	Future Scope	41

REFERENCES	42
A SUPPLEMENTARY MATERIALS	43
A.1 Sample API Payloads	43
A.2 Sample Verification Outputs	44
A.3 Sample SQL Tamper Demonstration	44
A.4 Optional Screenshot Recommendation	45

TABLE NO.	TITLE	PAGE
	NO.	

LIST OF TABLES

3.1	Core data entities used in Health Chain.	10
5.1	Core tables used in HealthChain application database.	17
6.1	Engineering module responsibilities across implementation layers.	23
8.1	Threat-to-control mapping used in the implementation.	31
9.1	Deployment checklist for reproducible setup.	35
9.2	Representative test/evaluation matrix and outcomes.	37

FIGURE NO.	TITLE	PAGE
	NO.	

LIST OF FIGURES

1.1	Core integrity loop of the Health Chain system.	4
4.1	High-level architecture of the Health Chain platform.	11
4.2	Clinical record creation and integrity verification workflow.	12
5.1	Simplified relational dependency view for HealthChain operational tables. .	16
5.2	Suggested screenshot for Chapter 5: relational schema evidence.	18
5.3	End-to-end implementation flow from user action to blockchain-backed re- sponse.	19
6.1	Engineering request lifecycle with normal and failure branches.	23
6.2	Suggested screenshot for Chapter 6: modular backend organization evidence.	24
7.1	Verification decision flow used by the integrity engine.	27
7.2	Suggested screenshot set for Chapter 7 logic-engine evidence.	29
8.1	Suggested screenshot for Chapter 8: audit and accountability interface evi- dence.	32
8.2	Audit lifecycle from user action to compliance-visible trace.	32
9.1	Suggested screenshot for Chapter 9: deployment reproducibility evidence. .	35
9.2	Verification evidence used in testing and evaluation.	38
9.3	Suggested screenshot set for Chapter 9 discussion evidence.	39

LIST OF SYMBOLS AND ABBREVIATIONS

ABBREVIATION	EXPANSION
EHR	Electronic Health Records
RBAC	Role-Based Access Control
JWT	JSON Web Token
API	Application Programming Interface
SHA-256	Secure Hash Algorithm 256-bit
HIPAA	Health Insurance Portability and Accountability Act
TLS	Transport Layer Security
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 Background and Need

Healthcare organizations depend on accurate and trustworthy records for diagnosis, treatment planning, legal compliance, insurance processing, and continuity of care. Most hospital information systems are built on centralized relational databases that provide efficient querying and structured storage. However, centralized storage alone does not inherently provide tamper evidence. If records are modified at the database level through unauthorized access, misconfiguration, or insider abuse, the system may not detect the change without an additional integrity mechanism.

Clinical records are highly sensitive and legally significant. A single altered diagnosis or prescription note can affect treatment decisions and patient safety. In this context, record integrity is not only a technical issue but also an ethical and regulatory requirement. This motivates a design where routine EHR operations remain practical while data integrity becomes independently verifiable.

Health Chain addresses this need by integrating MySQL with blockchain-style hash chaining. The project avoids replacing the relational model; instead, it introduces a complementary verification layer that marks each clinical record with a deterministic SHA-256 hash and stores it as a linked block. Any later mismatch between current record content and blockchain hash is treated as tampering.

1.2 Problem Statement

The central problem addressed in this project is:

How can a practical doctor-facing EHR platform preserve the efficiency of a normalized MySQL workflow while adding a robust and auditable mechanism to detect post-entry tampering of medical records?

In routine healthcare systems, records are distributed across multiple normalized tables, and updates occur through user interfaces and backend APIs. This modularity improves maintainability but introduces challenges for integrity assurance:

1. record content is split across multiple tables and joins,
2. tampering may occur outside official application flows,
3. role-based access alone does not guarantee immutability,
4. audit logs help trace actions but do not independently prove record integrity.

Hence, the requirement is to build a system where integrity can be verified deterministically at any time, with clear secure-versus-tampered outcomes.

1.3 Project Objectives

This project is developed with the following objectives:

1. Design a full-stack EHR workflow for doctors, patients, and administrators.
2. Implement transactional storage of SOAP records in normalized MySQL tables.
3. Generate deterministic SHA-256 hashes for clinical records.
4. Build an append-only blockchain ledger linked by previous-hash references.
5. Provide per-record and chain-level verification APIs for tamper detection.
6. Maintain HIPAA-oriented operational traceability using audit logging.

7. Support scheduling, dashboard analytics, and role-based access as real-world workflow features.

1.4 Scope and Boundaries

1.4.1 Included in Scope

1. Web-based EHR workflow for patient records and appointments.
2. Role-based login for patient, doctor, and admin users.
3. Deterministic record hashing and blockchain block creation.
4. Verification APIs and blockchain explorer views.
5. Audit trail generation for key user actions.
6. Field-level encryption for selected profile attributes.

1.4.2 Out of Scope

1. Public decentralized blockchain deployment.
2. Smart-contract execution on external chains.
3. Full production HIPAA certification process.
4. Integration with national EHR interoperability standards.
5. High-availability cloud deployment and auto-scaling.

1.5 Contributions of the Work

1. A hybrid architecture that combines relational data handling with blockchain-based integrity checking.

2. Deterministic record serialization and hash generation strategy for reproducible verification.
3. Practical clinical workflow implementation with SOAP entry, timeline access, and scheduling.
4. End-to-end tamper detection capability with clear SECURE/TAMPERED outcomes.
5. Integrated compliance-minded features including audit logs and access controls.

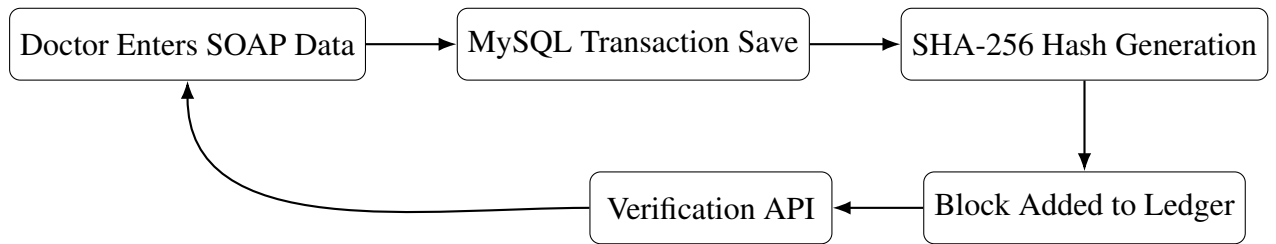


Figure 1.1: Core integrity loop of the Health Chain system.

1.6 Report Organization

The remainder of this report is structured as follows:

1. **Chapter 2** presents the literature survey and related work.
2. **Chapter 3** defines requirements, constraints, and technical specification.
3. **Chapter 4** explains architecture, modules, and system design.
4. **Chapter 5** details implementation methodology and core flows.
5. **Chapter 6** covers testing, validation, and observed results.
6. **Chapter 7** presents API design and end-to-end data flow analysis.
7. **Chapter 8** details security, privacy, and compliance-oriented controls.
8. **Chapter 9** explains deployment workflow, user operation guide, and viva preparation.
9. **Chapter 10** concludes with limitations and future enhancement directions.

CHAPTER 2

LITERATURE SURVEY

2.1 Traditional EHR Systems and Integrity Challenges

Conventional EHR systems are mostly built on centralized database architectures. They are optimized for structured storage, rapid retrieval, and complex clinical reporting through SQL. These systems support high transaction volumes and provide mature tooling for backup and recovery. Despite these strengths, many implementations rely on trust in access controls and administrative governance without embedding cryptographic tamper evidence in the data lifecycle.

Literature on healthcare security repeatedly identifies insider threat and privileged misuse as key risks. Even when role-based controls are present, database-level changes can be difficult to detect unless there is independent integrity instrumentation. This motivates cryptographic verification techniques that complement, rather than replace, mainstream EHR platforms.

2.2 Blockchain in Healthcare Data Management

Blockchain-inspired methods have been investigated for medical data provenance, consent traceability, and integrity assurance. Most studies emphasize immutability through linked blocks and cryptographic hashing, where any modification to prior data invalidates downstream chain consistency.

In practical healthcare software engineering, fully decentralized blockchain adoption often

faces constraints such as operational complexity, cost, and integration friction with existing DBMS workflows. As a result, hybrid designs are increasingly explored: critical record fingerprints are anchored to a ledger while primary data remains in relational storage.

Health Chain follows this hybrid direction by storing complete clinical records in MySQL and preserving record fingerprints in a local chain. This strategy keeps query efficiency intact and introduces tamper detectability through deterministic hash comparison.

2.3 Hash-Based Tamper Detection Approaches

Cryptographic hash functions are widely used for integrity verification because small input changes produce large output differences. In medical record systems, deterministic record serialization is essential: identical logical content must always produce the same hash. If formatting is inconsistent, verification may fail even without tampering.

The reviewed approaches suggest that reliable tamper detection depends on:

1. stable canonical representation of record fields,
2. consistent ordering of serialized data,
3. secure hash function choice,
4. reliable mapping between stored records and integrity anchors.

These principles are adopted in this project through a fixed SOAP text composition and SHA-256 hashing pipeline.

2.4 Audit Logging and Compliance Perspective

Healthcare regulation frameworks emphasize accountability and traceability. Audit logs capture who accessed what data and when actions occurred. However, logs alone do not prove that current record content is unchanged from original submission.

Therefore, best practice in security architecture combines event logging with cryptographic integrity checks. Health Chain reflects this integrated view by maintaining action-level

audit trails alongside verification endpoints and chain validation routines.

2.5 Research Gap and Motivation

A clear practical gap exists between academic blockchain proposals and deployable student-level healthcare systems. Many studies discuss theoretical immutability but do not provide complete end-to-end implementations involving:

1. normalized healthcare schema,
2. transactional clinical inserts,
3. role-based APIs,
4. blockchain verification endpoints,
5. scheduler and dashboard workflows.

This project is motivated by that implementation gap. It demonstrates a realistic, full-stack prototype where integrity verification is embedded into normal EHR operations rather than treated as a separate demonstration module.

CHAPTER 3

PROJECT REQUIREMENTS AND SPECIFICATION

3.1 Functional Requirements

The system shall satisfy the following functional requirements:

1. Register and authenticate users with role-specific access.
2. Allow doctors to create full SOAP records for selected patients.
3. Persist visit data in normalized relational tables using transactions.
4. Generate deterministic SHA-256 hashes for every created visit record.
5. Append each record hash as a new block linked to prior blockchain state.
6. Provide record verification endpoint to classify SECURE or TAMPERED status.
7. Provide chain status endpoint for full blockchain validity checks.
8. Support patient profile retrieval and timeline visualization.
9. Support appointment booking with conflict detection and status updates.
10. Capture audit logs for key security and clinical actions.
11. Provide admin interfaces for doctor account management.

3.2 Non-Functional Requirements

1. **Integrity:** deterministic and repeatable record verification.
2. **Security:** password hashing, role-based authorization, and token authentication.
3. **Reliability:** transactional inserts to avoid partial record state.
4. **Performance:** responsive API operations for standard clinic usage.
5. **Maintainability:** modular backend controllers and organized frontend components.
6. **Usability:** clear workflow for doctors and administrators.
7. **Auditability:** immutable-style evidence through chain and logs.

3.3 Input-Output Contracts

Primary Inputs

1. Authentication payloads (username, password).
2. Clinical record payloads (SOAP fields, vitals, severity, follow-up).
3. Scheduler payloads (patient, date, time, reason).
4. Verification requests (visit identifier).

Primary Outputs

1. JWT token and role metadata after login.
2. Persisted record identifier and blockchain hash after record creation.
3. Verification verdict for requested record.
4. Paginated datasets for patients, records, appointments, and audit logs.
5. Chain validity summary for blockchain monitoring.

3.4 Data Model Overview

The database includes authentication tables, profile tables, visit tables, appointment table, and audit logs. Visits are normalized into visits, vitals, diagnoses, and prescriptions. This separation reduces redundancy and keeps data management structured, while the blockchain hash binds these distributed fields into a single integrity fingerprint.

Table 3.1: Core data entities used in Health Chain.

Entity	Category	Purpose
users	Authentication	Stores credentials and role mapping
patients, doctors	Profile	Stores domain-specific identity information
visits, vitals, diagnoses, prescriptions	Clinical	Stores normalized SOAP-linked encounter data
audit_logs	Compliance	Stores action trace events
appointments	Scheduling	Stores doctor-patient slot planning records
blockchain_data.json	Integrity	Stores linked block metadata and hashes

CHAPTER 4

SYSTEM ARCHITECTURE AND DESIGN

4.1 High-Level Architecture

The system is implemented as a three-tier architecture with a React-based client, Express API server, and MySQL database, augmented by a blockchain ledger file for integrity meta-data. APIs act as the contract boundary between user interfaces and data services.

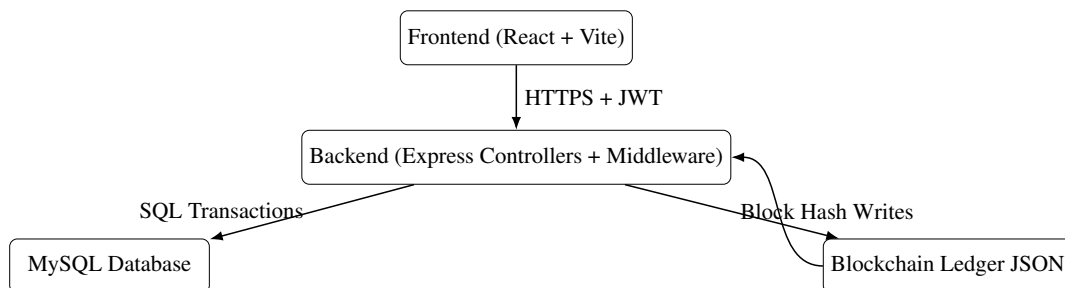


Figure 4.1: High-level architecture of the Health Chain platform.

4.2 Backend Module Design

The backend is organized into middleware, controllers, models, and utility layers.

1. **Middleware layer:** token verification and role authorization.
2. **Controller layer:** auth, records, appointments, dashboard, and admin flows.
3. **Model layer:** SQL transactions, record retrieval, chain operations.
4. **Utility layer:** hashing and sensitive-field encryption support.

This separation improves maintainability and allows integrity logic to remain explicit in record workflows.

4.3 Record Creation and Verification Flow

Figure 4.2 presents the operational data flow for record creation and verification.

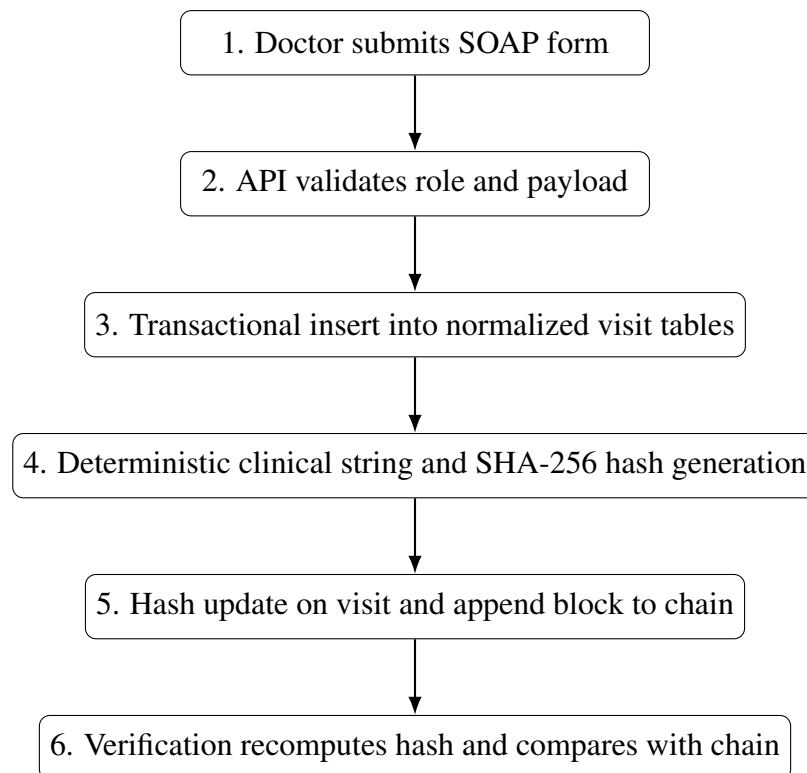


Figure 4.2: Clinical record creation and integrity verification workflow.

4.4 Security and Compliance Design

Security design includes authentication, authorization, data protection, and accountability.

1. Passwords are stored using bcrypt hashes.
2. JWT tokens carry user identity and role claims.
3. Role checks are enforced per API endpoint.
4. Sensitive profile fields can be encrypted at rest.

5. Audit logs record critical actions including record creation and verification.

CHAPTER 5

DATABASE DESIGN AND DATA FLOW

5.1 Database Overview

The database is the central operational backbone of HealthChain Bridge because all critical workflows depend on reliable persisted state: authentication, patient profile access, clinical record creation, appointment scheduling, integrity verification, and compliance logging.

The design follows a hybrid approach:

1. **Relational storage (MySQL):** canonical healthcare entities and transactional operations.
2. **Integrity ledger (JSON blockchain):** tamper-evident hash anchoring linked to record identifiers.

This ensures that day-to-day clinical operations keep SQL efficiency while integrity checks remain independently verifiable.

In practical terms, this hybrid choice addresses a common healthcare software gap. Traditional systems are strong at structured retrieval but weak at proving post-write immutability. Pure decentralized blockchain systems provide stronger immutability semantics but introduce operational complexity for normal hospital or clinic workflows. HealthChain Bridge therefore takes a middle path: keep high-usability transactional storage in MySQL and attach cryptographic trust signals through a deterministic ledger layer.

From an engineering perspective, this also improves maintainability. Database administrators can continue using familiar SQL tooling for backup, migration, and query optimization,

while integrity validation can evolve independently as a service concern. This separation is important for final-year systems because it demonstrates both conceptual correctness and implementation pragmatism.

5.2 Core Relational Schema Design

The schema is mapped to real system workflows rather than abstract entities. Major tables include:

1. `users`: authentication identity and role.
2. `patients` and `doctors`: profile-level role extensions.
3. `visits`, `vitals`, `diagnoses`, `prescriptions`: normalized SOAP persistence.
4. `appointments`: scheduling lifecycle.
5. `audit_logs`: compliance traceability.

The normalization strategy supports clean joins for patient timeline retrieval and prevents update anomalies in clinical detail storage.

Another design objective was to preserve semantic clarity between encounter-level and measurement-level data. For example, a visit represents the clinical context (who, when, why), while vitals and diagnosis represent structured observations linked to that context. This mapping avoids overloading a single table with mixed concerns and helps both analytics and audit review workflows.

5.3 Transactional Database Reliability

Clinical record insertion is treated as an atomic multi-table operation. The backend creates visit, vitals, diagnosis, and prescription rows inside one transaction, so no partial record state is committed.

This choice improves both clinical reliability and engineering correctness:

1. failed intermediate write $\Rightarrow$ full rollback,
2. successful commit $\Rightarrow$ complete and query-consistent record set.

This transactional behavior directly supports integrity validation quality. If record fragments were partially committed, a later verification operation could fail for the wrong reason (incomplete data rather than tampering). By enforcing atomic writes, the project ensures that mismatch signals remain meaningful and interpretable.

5.4 Database Relationship and Access Path

Role-specific query paths are intentionally separated:

1. doctors/admins can query broader patient and operational datasets,
2. patient access is constrained to self-scoped records and status views,
3. admin modules use dedicated joins for doctor account management.

This creates predictable access boundaries at query level, not only at UI level.

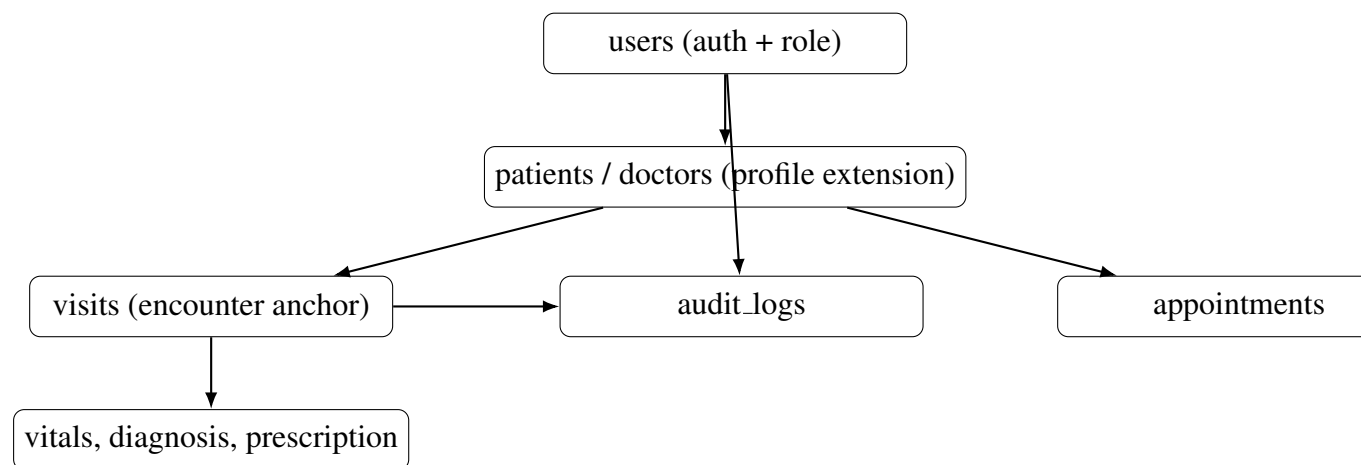


Figure 5.1: Simplified relational dependency view for HealthChain operational tables.

Table 5.1: Core tables used in HealthChain application database.

Table	Type	Role in Workflow
users	Auth core	Login identity, hashed password, role claims
patients	Profile	Patient demographic and medical profile context
doctors	Profile	Clinician identity and specialty mapping
visits	Clinical core	Encounter-level SOAP anchor and hash reference
vitals	Clinical detail	Objective measurements linked to visit
diagnoses	Clinical detail	Assessment and severity linked to visit
prescriptions	Clinical detail	Plan/medication instructions linked to visit
appointments	Scheduling	Date/time/status-based consultation planning
audit_logs	Compliance	Immutable action-level operational trace

5.5 Data Flow from Record Creation to Verification

The key database-centric flow in this project is the lifecycle from record insert to tamper validation:

1. write normalized SOAP tables transactionally,
2. build deterministic canonical SOAP string,
3. generate SHA-256 hash and store in relational record,
4. append hash fingerprint to blockchain ledger with previous-hash link,
5. on verification, reconstruct current DB content and compare hashes.

This ensures database state and ledger state remain cross-checkable at any point in time.

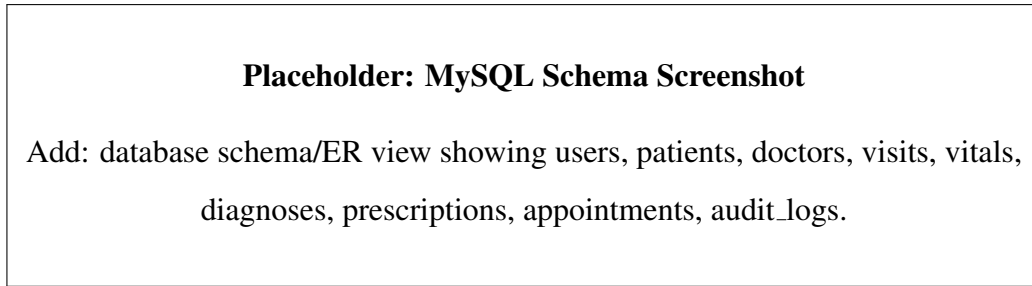


Figure 5.2: Suggested screenshot for Chapter 5: relational schema evidence.

5.6 Database and Data-Flow Summary

Chapter 5 established the project as a database-grounded healthcare system where trust is strengthened through cryptographic verification. The next chapter focuses on software engineering implementation workflows that operationalize these data and integrity designs in production-style modules.

5.7 Implementation Flow Diagram

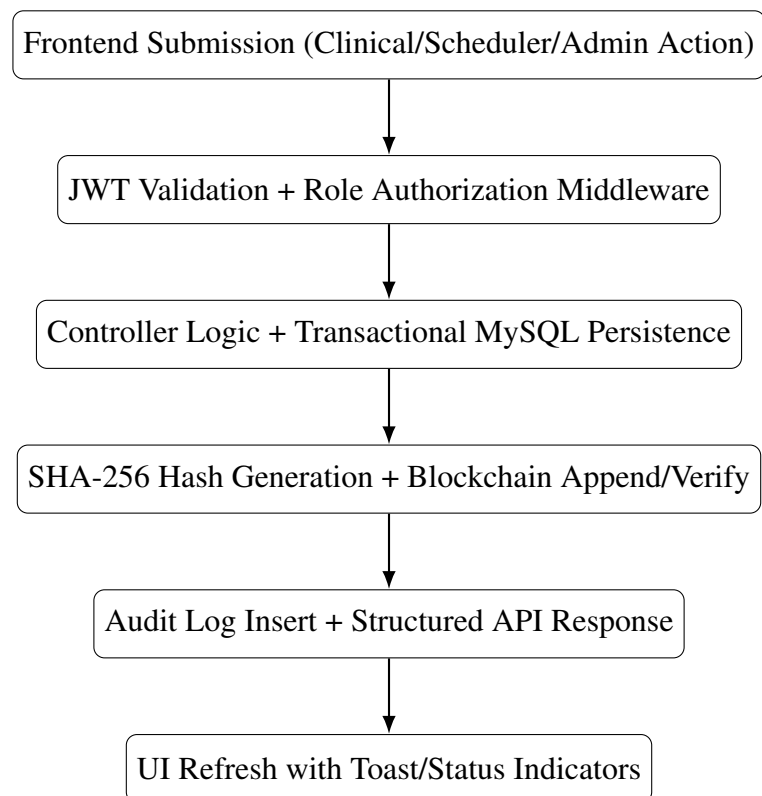


Figure 5.3: End-to-end implementation flow from user action to blockchain-backed response.

CHAPTER 6

SOFTWARE ENGINEERING IMPLEMENTATION WORKFLOW

6.1 Engineering Environment and Stack Composition

After establishing data architecture, the implementation was engineered as a modular full-stack workflow where each layer has a clear responsibility.

1. **Client engineering:** React + Vite route-based UI with role-aware navigation and reusable components.
2. **API engineering:** Express controllers with middleware-driven security and validation.
3. **Model engineering:** encapsulated data and blockchain operations to avoid business logic duplication.
4. **Utility engineering:** isolated hashing and encryption helpers for deterministic reuse.

The engineering approach intentionally favors separation of concerns over shortcut coding patterns. UI pages are organized by role and workflow, controllers are scoped by business capability, and shared helpers reduce repeated crypto/security logic. This structure was chosen to make debugging, testing, and viva explanation easier, since each module has a single dominant responsibility.

6.2 Backend Engineering Workflow

The backend implementation follows a clean request pipeline:

1. route mapping,
2. token and role middleware,
3. controller orchestration,
4. model/database execution,
5. audit and response emission.

This pattern improves maintainability because each layer is testable and change-scoped.

A key implementation quality point is that policy checks happen before business execution. In other words, authentication and authorization failures are resolved at middleware stage, not deep inside controller branches. This reduces hidden edge cases and makes negative-path behavior easier to reason about.

6.2.1 Controller-Oriented Business Modules

The implementation is engineered as domain modules rather than one large controller:

1. authentication and account flow,
2. clinical record management and verification,
3. appointment scheduling and conflict control,
4. dashboard analytics aggregation,
5. admin provisioning workflows.

This modularity reduces coupling and improves feature extensibility.

6.2.2 Codebase Organization and Reuse Strategy

Reusable components and utilities are treated as first-class engineering assets in the implementation. On the frontend, shared elements such as patient search, pagination, records cards, and layout controls reduce UI duplication. On the backend, utility abstractions for hashing, encryption, and role middleware provide consistent behavior across route families.

This reuse strategy reduces regression risk. When security or validation behavior is updated in one shared module, all dependent routes or pages inherit the improvement without repeated edits.

6.2.3 Reliability and Consistency Engineering

Reliability is enforced through transactional persistence and deterministic integrity operations. Critical write workflows are atomic, and integrity artifacts are generated from canonical serialized input, making verification behavior reproducible across environments.

6.2.4 Failure Handling and Recovery Paths

Implementation quality is also reflected in predictable failure handling:

1. validation errors return explicit client-consumable messages,
2. conflict scenarios use semantic HTTP 409 responses,
3. unauthorized paths terminate early with 403 responses,
4. verification failures return safe status classifications instead of silent success.

These choices improve operational reliability and reduce ambiguity during demonstrations and testing.

6.3 Cross-Layer Integration Flow

The project engineering focus is not only feature presence, but integration consistency across modules:

1. frontend submits authenticated payloads,
2. middleware enforces access policy,
3. controllers orchestrate model operations,
4. relational and blockchain states are updated coherently,
5. audit logging preserves post-action traceability.

Table 6.1: Engineering module responsibilities across implementation layers.

Module	Layer	Engineering Role
apiRoutes + middleware	Integration layer	Route binding, JWT validation, RBAC policy
authController	Business layer	Account lifecycle and token issuance
recordController	Business layer	Multi-table clinical flow + verify orchestration
appointmentController	Business layer	Conflict-safe scheduling lifecycle
Records model	Data layer	Transactional DB write/read encapsulation
Blockchain model	Integrity layer	Linked-block persistence and validation
AuditLog model	Compliance layer	Action trace persistence for accountability

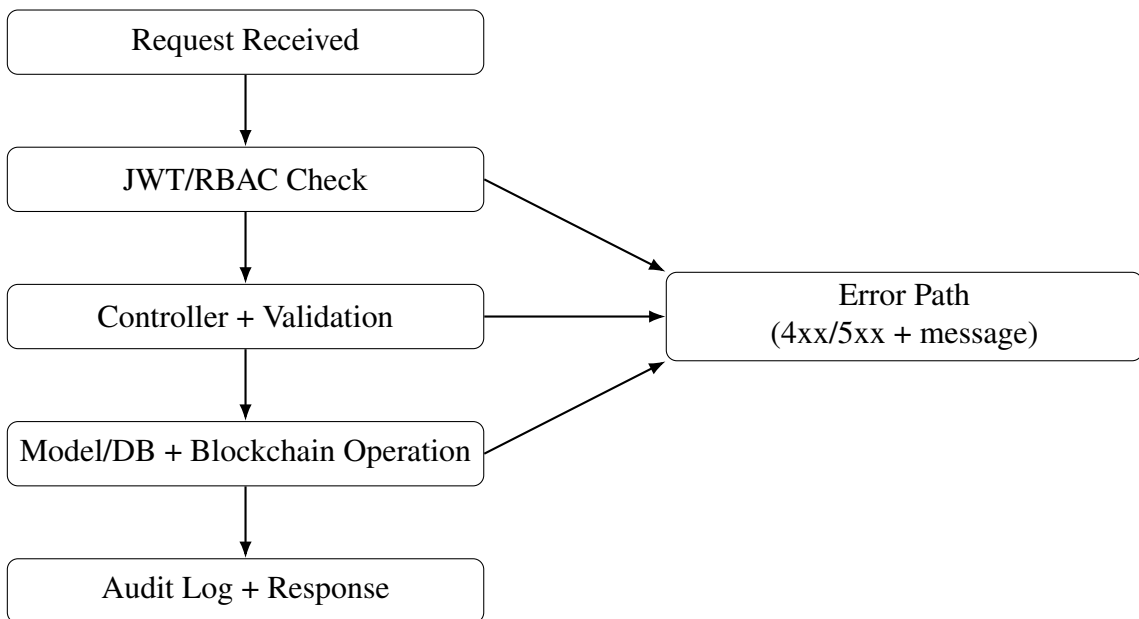


Figure 6.1: Engineering request lifecycle with normal and failure branches.

6.4 Engineering Discussion

The implementation demonstrates practical software engineering progression from architecture to reliable workflows. Frontend concerns were established in earlier chapters; this chapter emphasized backend orchestration, module separation, integration quality, and reliability choices that make the system maintainable and extensible.

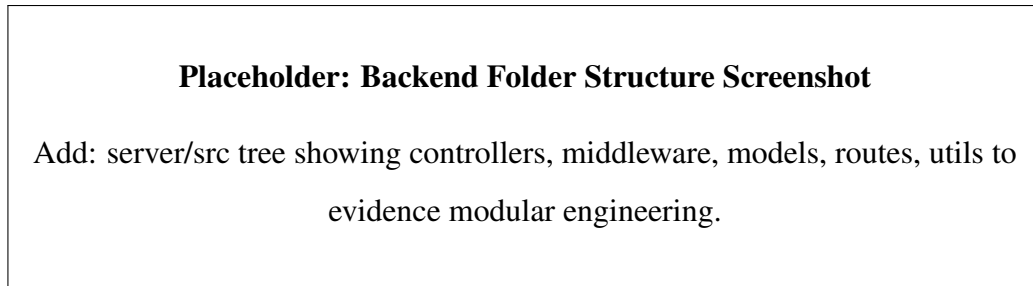


Figure 6.2: Suggested screenshot for Chapter 6: modular backend organization evidence.

CHAPTER 7

LOGIC DESIGN AND INTEGRITY ENGINE

7.1 Chapter Purpose and Logic Scope

This chapter explains the core logic that gives HealthChain Bridge its technical value beyond interface and database implementation. While earlier chapters covered data structures and software engineering workflow, this chapter focuses on decision rules and algorithmic behavior: how record content is canonicalized, how hashes are generated, how blockchain linkage is created, how verification verdicts are computed, and how scheduling and role-access decisions are made.

The main idea is that trust in records should not depend only on access control; it should be computable and re-checkable through deterministic logic.

7.2 Deterministic SOAP Canonicalization Logic

The integrity pipeline begins by transforming clinical record content into a canonical string representation. This step is crucial because hash verification is only meaningful when the same logical record always produces the same input string.

The implemented canonical pattern follows the SOAP structure:

1. Subjective line (chief complaint and relevant narrative),
2. Objective line (vitals in normalized order),
3. Assessment line (diagnosis/severity context),

4. Plan line (prescription and follow-up instruction).

Whitespace trimming and stable field ordering are enforced to avoid accidental hash drift caused by formatting differences. This means comparison failures are more likely to indicate genuine content changes rather than serialization noise.

7.3 Hash Generation and Block Anchoring Logic

After canonicalization, the system computes a SHA-256 digest and stores it as the record fingerprint. A new blockchain block is then created with index, timestamp, record id, data hash, previous hash, and current hash.

The block hash follows a deterministic composition of block fields. Conceptually:

$$\text{currentHash} = H(\text{index} \parallel \text{timestamp} \parallel \text{recordId} \parallel \text{dataHash} \parallel \text{previousHash})$$

where H is SHA-256 and $\parallel$ denotes concatenation in fixed order.

This chaining ensures that modifying one block affects all downstream hash relationships, making tampering detectable.

7.4 Verification Decision Engine

The verification logic is a deterministic decision engine rather than a heuristic check. Its runtime sequence is:

1. fetch relational record components for the target visit,
2. rebuild canonical SOAP string using the same formatting logic,
3. recompute SHA-256 hash,
4. fetch blockchain-stored hash for the same record id,
5. compare recomputed hash with anchored hash,

6. emit verdict: SECURE or TAMPERED.

Decision rule:

$$\text{status} = \begin{cases} \text{SECURE}, & \text{if } h_{db} = h_{chain} \\ \text{TAMPERED}, & \text{if } h_{db} \neq h_{chain} \end{cases}$$

This binary decision model provides clear interpretation for users and reviewers.

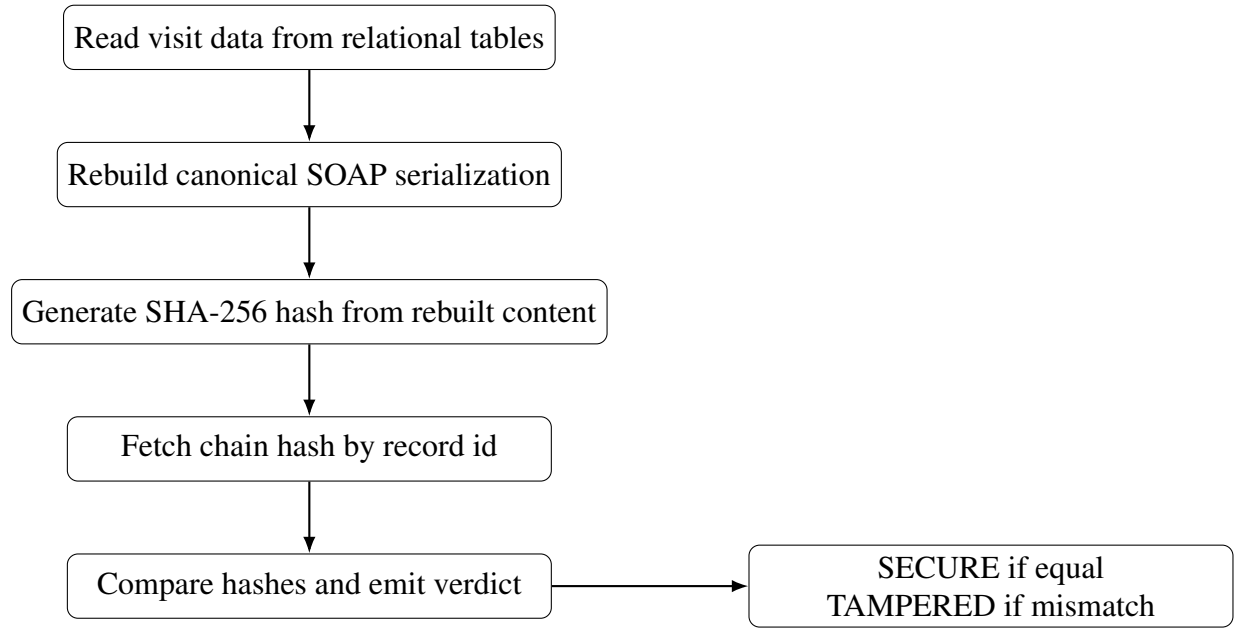


Figure 7.1: Verification decision flow used by the integrity engine.

7.5 Role and Access Decision Logic

Role handling is implemented as a two-stage policy decision:

1. identity validation (JWT token parsing and signature check),
2. route authorization (allowed role set per endpoint).

The decision is fail-closed: when token validation fails or role is outside allowed set, request execution is terminated before business logic. This prevents unauthorized paths from reaching sensitive operations.

7.6 Scheduler Conflict Decision Logic

Appointment creation uses deterministic overlap rules to avoid double-booking behavior for doctors. On each new request, the system checks nearby timeslots in the same date window for conflict. If overlap exists within configured tolerance, the booking decision becomes a conflict rejection (HTTP 409); otherwise, insertion proceeds.

This logic improves practical usability by preventing inconsistent calendars and making rejection reasons explicit to the user interface.

7.7 Error and Recovery Logic

The logic layer includes explicit failure semantics:

1. transaction rollback for partial clinical write failure,
2. structured error payloads for validation/authorization failures,
3. deterministic verify fallback when record or block lookup is unavailable,
4. audit event generation for critical success/failure actions.

These controls ensure that system behavior remains explainable in both normal and exceptional paths.

7.8 Logic Validation Summary

The chapter demonstrates that HealthChain Bridge is not only a data-entry platform; it is a logic-driven integrity system. Deterministic canonicalization, cryptographic hashing, block chaining, explicit decision rules, and policy-driven access control together create the project's core technical contribution.

Placeholder: Logic Evidence Screenshot Set

Add two images: (1) Clinical Workstation showing live hash preview, (2) Blockchain Explorer showing matching verification state for a new record.

Figure 7.2: Suggested screenshot set for Chapter 7 logic-engine evidence.

CHAPTER 8

SECURITY, PRIVACY, AND COMPLIANCE ANALYSIS

8.1 Security Objectives

Security and privacy implementation in HealthChain targets four concrete goals:

1. authenticated and role-restricted access,
2. confidentiality of credentials and sensitive profile data,
3. detectable integrity violation for medical records,
4. traceable audit events for accountability and review.

8.2 Threat Model Overview

The project threat model includes realistic application-level risks:

1. credential misuse and unauthorized login attempts,
2. role escalation and endpoint abuse,
3. direct database-level modification of records,
4. inconsistent state due to partial writes,
5. untracked access to sensitive operations.

Table 8.1: Threat-to-control mapping used in the implementation.

Threat	Control Mechanism	Residual Risk
Credential compromise	bcrypt + JWT expiry window	Weak password hygiene on user side
Unauthorized endpoint use	role middleware checks	Token theft on insecure client device
DB-level record alteration	deterministic hash + blockchain verification	Simultaneous compromise of DB and ledger storage
Partial clinical insert failure	SQL transaction rollback	Infrastructure/DB outage scenarios
Untraceable action history	centralized audit log capture	Local log storage durability assumptions

8.3 Authentication and Authorization Controls

Authentication uses JWT bearer tokens issued after bcrypt-based password verification. Authorization is implemented with route-level RBAC middleware that enforces role permissions before controller execution.

Together, this provides identity validation and permission validation as separate control layers.

The layered approach reduces security drift over time. As new routes are added, the same middleware stack can be applied with minimal effort, ensuring that security behavior remains uniform across modules.

8.4 Confidentiality and Data Protection Controls

Confidentiality controls include:

1. secure password hashing (bcrypt),
2. optional HTTPS for local secure transport,
3. optional MySQL TLS mode,
4. AES-256-GCM encryption utility for sensitive patient profile fields,
5. role-scoped data access policies for patients, doctors, and admins.

8.5 Integrity and Non-Repudiation Perspective

HealthChain does not use a public consensus blockchain; instead, it applies an in-application immutable-link concept. Each clinical record has a deterministic hash anchored into a chained block structure. Re-verification compares current DB state to anchored hash fingerprints.

This provides practical tamper evidence and strengthens non-repudiation for healthcare record modifications.

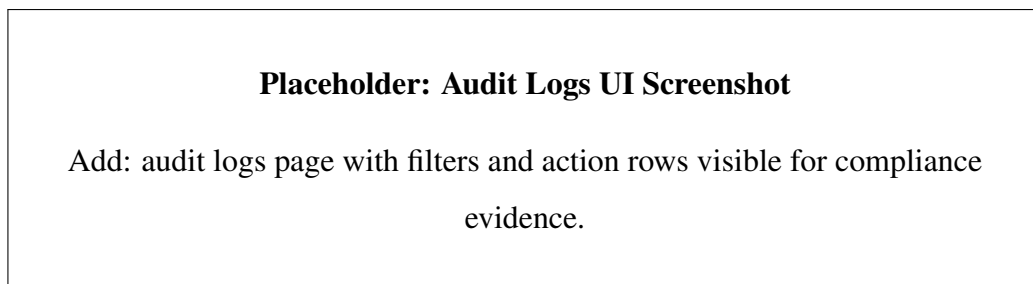


Figure 8.1: Suggested screenshot for Chapter 8: audit and accountability interface evidence.

8.6 Audit and Compliance Perspective

The audit log subsystem records critical actions (login, record create/view, verification, administrative operations) with user, action type, details, and timestamp.

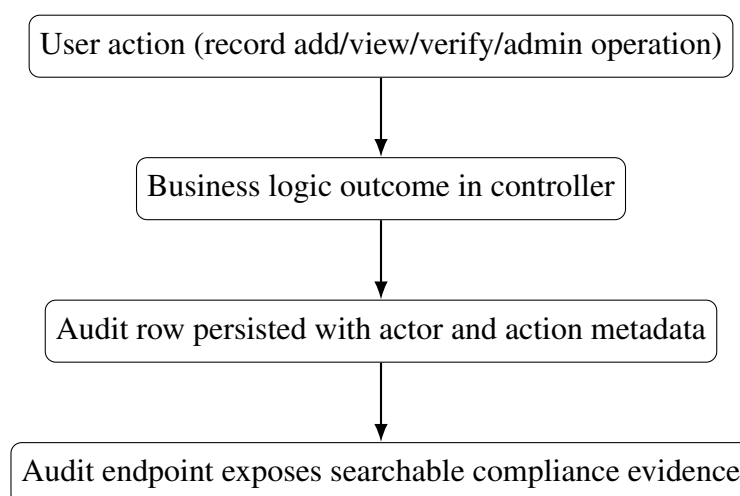


Figure 8.2: Audit lifecycle from user action to compliance-visible trace.

8.7 Limitations of Current Security Scope

The current implementation is designed for educational and prototype deployment. It does not claim full production compliance certification and still requires enterprise hardening in key management, immutable remote logging, and regulated governance controls.

8.8 Security Enhancement Roadmap

1. key rotation and secure vault-backed key lifecycle,
2. optional MFA and stronger password policy enforcement,
3. anomaly alerts for repeated mismatch or suspicious access patterns,
4. periodic signed chain snapshots,
5. administrative security dashboards and audit export pipelines.

CHAPTER 9

DEPLOYMENT, TESTING, EVALUATION, AND DISCUSSION

9.1 Deployment Overview

The project is deployed as a coordinated three-service setup: MySQL database, Express backend API, and React frontend.

1. Start MySQL (Docker or local instance) and initialize schema.
2. Configure backend environment and run API service.
3. Configure frontend API endpoint and run client service.
4. Validate blockchain status and create a test record.
5. Confirm audit logs and scheduler flows are operational.

This sequence is intentionally ordered to reduce debugging complexity. Database readiness is validated first, then backend service correctness, and finally frontend integration. By enforcing this progression, deployment failures are isolated to a narrower layer and can be corrected quickly during demonstration setup.

9.2 Configuration Checklist

Table 9.1: Deployment checklist for reproducible setup.

Item	Required	Verification Method
Schema applied to <code>healthchain_db</code>	Yes	Required tables visible and queryable
Backend env configured (.env)	Yes	API boots without runtime config errors
Frontend API URL configured	Yes	Login and record retrieval succeed
Blockchain file writable	Yes	Block appended after creating a record
Audit logging path active	Yes	New entries visible in <code>/api/audit-logs</code>
TLS configuration (optional)	Recommended	API/client reachable via HTTPS locally

9.3 Demo Data and Reset Workflow

The implementation includes automated demo generation scripts for review readiness:

1. run schema creation,
2. start backend,
3. run demo generator for doctors/patients/records/appointments,
4. apply seed profile updates,
5. verify blockchain length and sample record integrity.

This workflow ensures repeatable demonstration state before viva or evaluation.



Figure 9.1: Suggested screenshot for Chapter 9: deployment reproducibility evidence.

9.4 Testing, Evaluation, and Discussion Overview

After deployment readiness is established, validation moves to behavior correctness and trust verification. The testing scope is not limited to API responses; it also checks role-policy enforcement, deterministic integrity outcomes, scheduler conflict handling, and audit trace visibility. This integrated approach demonstrates that the system is both runnable and reliably verifiable.

9.5 Testing Methodology

Testing was carried out at unit-like function scope, API integration scope, and end-to-end workflow scope to verify that the HealthChain implementation behaves correctly as an engineering system rather than only as a UI demo.

1. functional API behavior testing,
2. middleware and role enforcement testing,
3. blockchain and tamper-detection testing,
4. scheduler conflict and status-transition testing,
5. audit and observability testing.

9.6 Representative Validation Matrix

Table 9.2: Representative test/evaluation matrix and outcomes.

ID	Test Case	Expected Result	Observed Result
T1	Valid doctor login	JWT + role claims returned	Passed
T2	Restricted endpoint by wrong role	HTTP 403 Access Denied	Passed
T3	Full SOAP record insert path	Transaction commit + hash + block	Passed
T4	Verify untouched record	SECURE status	Passed
T5	Tamper DB field then verify	TAMPERED status	Passed
T6	Overlapping appointment booking	HTTP 409 conflict	Passed
T7	Audit filter and pagination query	Correct filtered result set	Passed
T8	Chain status validation endpoint	Valid chain in untampered state	Passed

9.7 Tamper-Evidence Evaluation Scenario

The critical engineering claim is integrity breach detection. Validation sequence:

1. create a record through official clinical flow,
2. verify and observe SECURE state,
3. modify stored clinical data directly in DB,
4. re-run verification endpoint,
5. observe TAMPERED verdict.

This confirms independent integrity instrumentation beyond access-control assumptions.

Beyond correctness, this scenario is useful as a communication artifact for external evaluators. It demonstrates that the project can distinguish between normal clinical updates and unauthorized post-write modifications, which is the central trust objective of HealthChain Bridge.

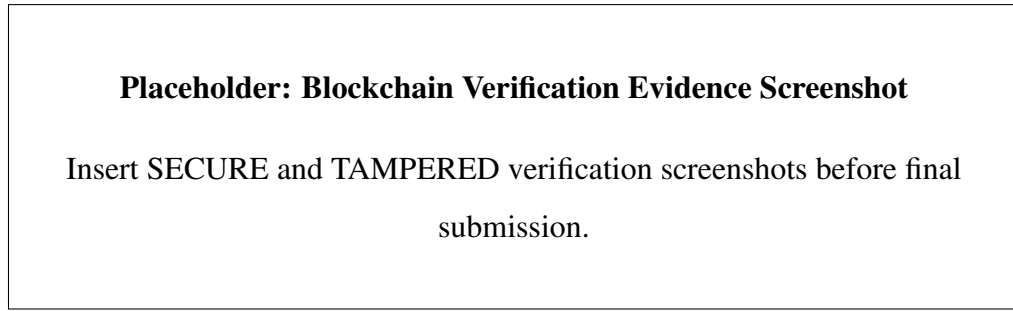


Figure 9.2: Verification evidence used in testing and evaluation.

9.8 Discussion of Results

9.8.1 Positive Outcomes

1. modular controllers and middleware produced stable workflow behavior,
2. transactional persistence prevented partial clinical state corruption,
3. blockchain-anchored hashing provided clear tamper evidence,
4. audit logs improved operational accountability and explainability.

9.8.2 Observed Engineering Constraints

1. file-based ledger is practical but not enterprise-distributed,
2. environment setup consistency strongly affects full harness reliability,
3. current test scope prioritizes functional correctness over load/performance stress.

9.8.3 Evaluation Summary

Overall results indicate that HealthChain Bridge achieves strong academic engineering quality: reproducible workflows, visible integrity checks, and coherent cross-layer integration. The next chapter concludes findings, limitations, and future direction.

Placeholder: Clinical Workstation + Blockchain Explorer Screenshots

Add two images: (1) Clinical Workstation form with live hash preview, (2) Blockchain Explorer showing verified block state.

Figure 9.3: Suggested screenshot set for Chapter 9 discussion evidence.

CHAPTER 10

CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Conclusion

HealthChain Bridge successfully demonstrates how blockchain-backed integrity checks can be integrated into a practical full-stack EHR/Doctor CRM system. The project delivers functional modules for role-based authentication, patient management, SOAP record capture, appointment scheduling, blockchain visualization, and compliance-oriented audit logging.

The key technical contribution is deterministic SHA-256 hashing of structured clinical content and linkage of each record to a chained block. This enables direct detection of unauthorized post-entry modifications while preserving the benefits of normalized relational storage in MySQL.

From an engineering perspective, the project combines frontend usability, backend modularity, transactional persistence, cryptographic integrity, and security-aware middleware in a coherent architecture suitable for final-year academic evaluation.

A strong outcome of the work is that integrity assurance is embedded into normal operational flow rather than treated as a disconnected add-on. Doctors continue using familiar documentation behavior, while the system silently adds verifiable trust instrumentation behind the scenes. This makes the design both technically meaningful and practically adoptable.

10.2 Limitations

Although the system meets its defined goals, several limitations remain:

1. ledger persistence is file-based and not yet distributed,
2. compliance implementation is educational rather than formally certified,
3. large-scale performance and resilience benchmarks are limited,
4. healthcare interoperability standards are not yet integrated.

10.3 Future Scope

Future versions can extend the platform in both technical and clinical utility dimensions:

1. migrate blockchain persistence to managed immutable/distributed infrastructure,
2. add digital signatures for clinician-authored records,
3. integrate standards-oriented interoperability gateways,
4. introduce stronger consent policies and governance controls,
5. add monitoring dashboards and anomaly detection pipelines,
6. perform load testing and production hardening,
7. package full deployment with container orchestration and secure CI/CD.

If implemented incrementally, these enhancements can transform the current prototype from strong academic proof-of-concept into a robust pilot-ready platform for integrity-sensitive clinical environments.

REFERENCES

1. S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.
2. M. Crosby, P. Pattanayak, S. Verma, and V. Kalyanaraman, “Blockchain Technology: Beyond Bitcoin,” *Applied Innovation Review*, no. 2, pp. 6–10, 2016.
3. A. Alammery, S. Alhazmi, M. Almasri, and S. Gillani, “Blockchain-Based Applications in Education: A Systematic Review,” *Applied Sciences*, vol. 9, no. 12, 2019.
4. A. Roehrs, C. A. da Costa, and R. da Rosa Righi, “OmniPHR: A Distributed Architecture Model to Integrate Personal Health Records,” *Journal of Biomedical Informatics*, vol. 71, pp. 70–81, 2017.
5. K. Peterson, R. Deeduvanu, P. Kanjamala, and K. Boles, “A Blockchain-Based Approach to Health Information Exchange Networks,” NIST Workshop, 2016.
6. Express.js Documentation, “Express 5 API Reference.” [Online]. Available: <https://expressjs.com/>
7. React Documentation, “React Developer Reference.” [Online]. Available: <https://react.dev/>
8. Node.js Documentation, “Crypto Module.” [Online]. Available: <https://nodejs.org/api/crypto.html>
9. MySQL Documentation, “MySQL 8.0 Reference Manual.” [Online]. Available: <https://dev.mysql.com/doc/>
10. OWASP Foundation, “JSON Web Token Cheat Sheet.” [Online]. Available: <https://cheatsheetseries.owasp.org/>

APPENDIX A

SUPPLEMENTARY MATERIALS

A.1 Sample API Payloads

The following payloads are representative examples used during implementation and validation.

A.1 Register Patient Payload

```
POST /api/register
{
  "username": "patient_demo",
  "password": "Demo@123",
  "role": "patient",
  "full_name": "Demo Patient",
  "age": 32,
  "gender": "Female"
}
```

A.2 Create Clinical Record Payload

```
POST /api/records
{
  "patient_id": 5,
  "subjective": "Headache for two days",
  "objective": "Mild fever, BP stable",
}
```

```
"assessment": "Viral febrile illness",
"plan": "Paracetamol and hydration",
"vitals": {
  "temperature": "99.8F",
  "blood_pressure": "120/80",
  "heart_rate": "82"
}
}
```

A.2 Sample Verification Outputs

A.3 Record Verification Response (Secure)

```
{
  "visit_id": 42,
  "status": "SECURE",
  "db_hash": "f6ac...91e2",
  "chain_hash": "f6ac...91e2"
}
```

A.4 Record Verification Response (Tampered)

```
{
  "visit_id": 42,
  "status": "TAMPERED",
  "db_hash": "73de...2a11",
  "chain_hash": "f6ac...91e2"
}
```

A.3 Sample SQL Tamper Demonstration

A.5 Controlled Tamper Query (For Demonstration Only)


```
UPDATE diagnoses  
SET diagnosis_text = 'Modified text for tamper demo'  
WHERE visit_id = 42;
```

This query is executed only in test environments to demonstrate hash mismatch detection.

A.4 Optional Screenshot Recommendation

The report includes a single outcome screenshot in Figure 9.2. This evidence image was chosen because it directly demonstrates the blockchain verification capability, which is the central contribution of the project.